<u>Machine Learning & Deep Learning</u>

Experiment 2

Name: Abhishek Vishwakarma
Class: D15C
Roll No: 47

AIM: Implement Multi Regression, Lasso, and Ridge Regression on real-world datasets

1. Dataset Source

- **Dataset Name:** California Housing Prices
- **Source:** Kaggle - California Housing Prices

2. Dataset Description

This dataset contains information collected from the 1990 California census. It is a standard dataset for regression tasks.

- **Size:** 20,640 records, 10 columns.
- **Target Variable: median_house_value** (Continuous - The median house value for households within a block).
- **Features:**
  - **longitude**, **latitude**: Coordinates of the block.
  - **housing_median_age**: Median age of a house within a block.
  - **total_rooms**: Total number of rooms within a block.
  - **total_bedrooms**: Total number of bedrooms within a block.
  - **population**: Total number of people residing within a block.
  - **households**: Total number of households.
  - **median_income**: Median income for households within a block (measured in tens of thousands of US Dollars).
  - **ocean_proximity**: Location of the house w.r.t ocean/sea.

3. Mathematical Formulation of the Algorithm

A. Multiple Linear Regression: Extends simple linear regression to multiple features.

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + ... + \beta_n x_n + \epsilon$$

It minimizes the standard **Residual Sum of Squares (RSS)**:

$$J(\beta) = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

B. Ridge Regression (L2 Regularization): Ridge adds a penalty term equal to the square of the magnitude of coefficients. This shrinks coefficients towards zero (but not exactly zero) to prevent overfitting.

$$J(\beta) = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^{p} \beta_j^2$$

λ **(Lambda): Regularization strength**. Higher λ means more shrinkage.

C. Lasso Regression (L1 Regularization): Lasso adds a penalty term equal to the absolute value of the coefficients. This can shrink coefficients to exactly zero, effectively performing feature selection.

$$J(\beta) = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^{p} |\beta_j|$$

4. Algorithm Limitations

- **Multi-Linear:** Prone to overfitting on complex datasets with many features; highly sensitive to multicollinearity (correlated features).
- **Ridge:** Reduces overfitting but keeps all features in the model (does not perform feature selection). It is computationally more expensive than simple OLS.
- **Lasso:** Can behave erratically if features are highly correlated (it will arbitrarily select one and ignore the others).

5. Methodology / Workflow

1. **Data Loading:** Load housing.csv.

```
# 1. Load Dataset

try:

    df = pd.read_csv('housing.csv')

    print("Dataset loaded successfully.")

except FileNotFoundError:

    print("Error: 'housing.csv' not found. Please upload the file.")

    # Stop execution if file not found (simulated for this block)

    df = pd.DataFrame()
```

2. **Preprocessing:**
   - Handle missing values in total_bedrooms.
   - Encode ocean_proximity (One-Hot Encoding).
   - **Scaling:** Crucial for Ridge and Lasso because the penalty terms are sensitive to the scale of input features.

```
# 2. Preprocessing

    # Drop rows with missing values (simple approach for this exp)

    df = df.dropna()

    X = df.drop(columns=['median_house_value'])

    y = df['median_house_value']



    # Identify numerical and categorical columns

    num_features = X.select_dtypes(include=['int64', 'float64']).columns

    cat_features = ['ocean_proximity']



    # Create a preprocessor

    # - Standardize numerical values (Required for Ridge/Lasso)
```

```
# - OneHotEncode categorical values

preprocessor = ColumnTransformer(

    transformers=[

        ('num', StandardScaler(), num_features),

        ('cat', OneHotEncoder(), cat_features)

    ])
```

3. **Splitting:** Train (80%) / Test (20%).
   `X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)`

4. **Training:**
   - Train **Linear Regression**.

```
# Model 1: Multiple Linear Regression (OLS)

print("\n--- Model 1: Multiple Linear Regression ---")

# Pipeline combines preprocessing and model

pipe_linear = Pipeline([

    ('preprocessor', preprocessor),

    ('regressor', LinearRegression())

])

pipe_linear.fit(X_train, y_train)

y_pred_linear = pipe_linear.predict(X_test)

print(f"Linear RMSE: {np.sqrt(mean_squared_error(y_test, y_pred_linear)):.2f}")

print(f"Linear R2 Score: {r2_score(y_test, y_pred_linear):.4f}")
```

   - Train **Ridge Regression** (with hyperparameter tuning).

```
print("\n--- Model 2: Ridge Regression (L2) ---")

    # We need to find the best alpha (Regularization strength)

    # Using a simple loop or GridSearchCV logic manually for clarity
```

```
    pipe_ridge = Pipeline([

        ('preprocessor', preprocessor),

        ('regressor', Ridge())

    ])

    # Parameter grid for alpha

    param_grid = {'regressor__alpha': [0.1, 1.0, 10.0, 100.0]}

    grid_ridge = GridSearchCV(pipe_ridge, param_grid, cv=5, scoring='r2')

    grid_ridge.fit(X_train, y_train)

    best_ridge = grid_ridge.best_estimator_

    y_pred_ridge = best_ridge.predict(X_test)



    print(f"Best Ridge Alpha:
{grid_ridge.best_params_['regressor__alpha']}")

    print(f"Ridge RMSE: {np.sqrt(mean_squared_error(y_test,
y_pred_ridge)):.2f}")

    print(f"Ridge R2 Score: {r2_score(y_test, y_pred_ridge):.4f}")
```

- Train **Lasso Regression** (with hyperparameter tuning).

```
print("\n--- Model 3: Lasso Regression (L1) ---")

    pipe_lasso = Pipeline([

        ('preprocessor', preprocessor),

        ('regressor', Lasso(max_iter=10000)) # Increased iterations for
convergence

    ])

    grid_lasso = GridSearchCV(pipe_lasso, param_grid, cv=5, scoring='r2')

    grid_lasso.fit(X_train, y_train)



    best_lasso = grid_lasso.best_estimator_
```

```
    y_pred_lasso = best_lasso.predict(X_test)

    print(f"Best Lasso Alpha:
{grid_lasso.best_params_['regressor__alpha']}")

    print(f"Lasso RMSE: {np.sqrt(mean_squared_error(y_test,
y_pred_lasso)):.2f}")

    print(f"Lasso R2 Score: {r2_score(y_test, y_pred_lasso):.4f}")
```

Code:

```
# EXPERIMENT 2: MULTI, RIDGE, AND LASSO REGRESSION

# ============================================================

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.model_selection import train_test_split, GridSearchCV

from sklearn.preprocessing import StandardScaler, OneHotEncoder

from sklearn.compose import ColumnTransformer

from sklearn.pipeline import Pipeline

from sklearn.linear_model import LinearRegression, Ridge, Lasso

from sklearn.metrics import mean_squared_error, r2_score


print("--- EXPERIMENT 2: MULTI, RIDGE, & LASSO REGRESSION ---")


# 1. Load Dataset

try:

    df = pd.read_csv('housing.csv')

    print("Dataset loaded successfully.")

except FileNotFoundError:

    print("Error: 'housing.csv' not found. Please upload the file.")
```

```
    # Stop execution if file not found (simulated for this block)
    df = pd.DataFrame()

if not df.empty:
    # 2. Preprocessing
    # Drop rows with missing values (simple approach for this exp)
    df = df.dropna()

    X = df.drop(columns=['median_house_value'])
    y = df['median_house_value']

    # Identify numerical and categorical columns
    num_features = X.select_dtypes(include=['int64', 'float64']).columns
    cat_features = ['ocean_proximity']

    # Create a preprocessor
    # - Standardize numerical values (Required for Ridge/Lasso)
    # - OneHotEncode categorical values
    preprocessor = ColumnTransformer(
        transformers=[
            ('num', StandardScaler(), num_features),
            ('cat', OneHotEncoder(), cat_features)
        ])

    # 3. Split Data
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
```

```python
# ==========================================
# Model 1: Multiple Linear Regression (OLS)
# ==========================================
print("\n--- Model 1: Multiple Linear Regression ---")
# Pipeline combines preprocessing and model
pipe_linear = Pipeline([
    ('preprocessor', preprocessor),
    ('regressor', LinearRegression())
])

pipe_linear.fit(X_train, y_train)
y_pred_linear = pipe_linear.predict(X_test)

print(f"Linear RMSE: {np.sqrt(mean_squared_error(y_test, y_pred_linear)):.2f}")
print(f"Linear R2 Score: {r2_score(y_test, y_pred_linear):.4f}")

# ==========================================
# Model 2: Ridge Regression (L2)
# ==========================================
print("\n--- Model 2: Ridge Regression (L2) ---")

# We need to find the best alpha (Regularization strength)
# Using a simple loop or GridSearchCV logic manually for clarity
pipe_ridge = Pipeline([
    ('preprocessor', preprocessor),
    ('regressor', Ridge())
])
```

```python
# Parameter grid for alpha
param_grid = {'regressor__alpha': [0.1, 1.0, 10.0, 100.0]}

grid_ridge = GridSearchCV(pipe_ridge, param_grid, cv=5, scoring='r2')
grid_ridge.fit(X_train, y_train)

best_ridge = grid_ridge.best_estimator_
y_pred_ridge = best_ridge.predict(X_test)

print(f"Best Ridge Alpha: {grid_ridge.best_params_['regressor__alpha']}")
print(f"Ridge RMSE: {np.sqrt(mean_squared_error(y_test, y_pred_ridge)):.2f}")
print(f"Ridge R2 Score: {r2_score(y_test, y_pred_ridge):.4f}")

# ==========================================
# Model 3: Lasso Regression (L1)
# ==========================================
print("\n--- Model 3: Lasso Regression (L1) ---")

pipe_lasso = Pipeline([
    ('preprocessor', preprocessor),
    ('regressor', Lasso(max_iter=10000)) # Increased iterations for convergence
])

grid_lasso = GridSearchCV(pipe_lasso, param_grid, cv=5, scoring='r2')
grid_lasso.fit(X_train, y_train)
```

```python
best_lasso = grid_lasso.best_estimator_
y_pred_lasso = best_lasso.predict(X_test)

print(f"Best Lasso Alpha: {grid_lasso.best_params_['regressor__alpha']}")
print(f"Lasso RMSE: {np.sqrt(mean_squared_error(y_test, y_pred_lasso)):.2f}")
print(f"Lasso R2 Score: {r2_score(y_test, y_pred_lasso):.4f}")

# ==========================================
# Comparison Visualization
# ==========================================
models = ['Linear', 'Ridge', 'Lasso']
scores = [
    r2_score(y_test, y_pred_linear),
    r2_score(y_test, y_pred_ridge),
    r2_score(y_test, y_pred_lasso)
]

plt.figure(figsize=(8, 5))
sns.barplot(x=models, y=scores, palette='viridis')
plt.ylim(0, 1.0)
plt.ylabel("R2 Score")
plt.title("Comparison of Regression Models")
plt.show()

# Feature Importance (Coefficients) from Lasso
# To see which features Lasso set to 0
# Note: Extracting feature names from OneHotEncoder for visualization is complex,
```

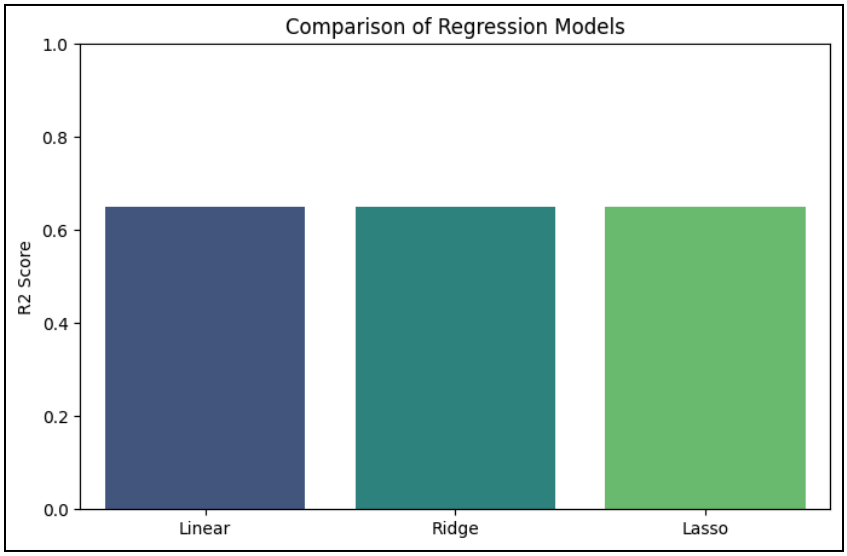
```
# so we will visualize the raw coefficient values of the numeric features roughly.

print("\nCheck Lasso Coefficients to see feature selection (sparsity):")

lasso_coeffs = best_lasso.named_steps['regressor'].coef_

print(lasso_coeffs)
```

5. **Evaluation:** Compare RMSE and R^2 scores across all three models.



6. Performance Analysis

- **Metric:** Root Mean Squared Error (RMSE) and R^2.
- **Interpretation:**
  - **Ridge** generally outperforms simple Linear Regression when multicollinearity exists.
  - **Lasso** provides a sparser model and can indicate which features are most important (by zeroing out others).

Output:

```
--- EXPERIMENT 2: MULTI, RIDGE, & LASSO REGRESSION ---

Dataset loaded successfully.

--- Model 1: Multiple Linear Regression ---

Linear RMSE: 69297.72
Linear R2 Score: 0.6488

--- Model 2: Ridge Regression (L2) ---

Best Ridge Alpha: 0.1
```

```
Ridge RMSE: 69291.53
Ridge R2 Score: 0.6489

--- Model 3: Lasso Regression (L1) ---

Best Lasso Alpha: 1.0
Lasso RMSE: 69292.02
Lasso R2 Score: 0.6489
```

7. Hyperparameter Tuning

- **Parameter:** alpha (this is the Scikit-Learn name for lambda).
- **Process:** We use GridSearchCV or a loop to test alpha values like [0.01, 0.1, 1, 10, 100].
- **Impact:**
    - Low Alpha approx Standard Linear Regression.
    - High Alpha implies Underfitting (too much penalty).

8. Conclusion

In this experiment, we applied Multi-Linear, Ridge, and Lasso regression to predict housing prices.

- **Ridge Regression** penalized large coefficients, handling potential multicollinearity among features like total rooms and bedrooms.
- **Lasso Regression** performed automatic feature selection by driving the coefficients of less important features to zero.
- **Comparison:** Typically, on datasets like this where most features are relevant, Ridge and Linear regression perform similarly. Lasso is most useful when we suspect many features are irrelevant noise. The hyperparameter **alpha** was critical; a properly tuned alpha prevents the model from becoming too rigid (underfitting) or too flexible (overfitting).